DEVOPS ENGINEERING

Build Faster . Deploy Smarter . Grow Together

People
Process
Tools
Culture
} = DevOps

① What is DevOps ?

DevOps is a combination of Development (Dev) and Operations (Ops) which aims to shorten the software development lifecycle and deliver high-quality software continuously with automation, collaboration and monitoring.

> "DevOps is not a tool it's a culture."

② Development + Operations

Development (Dev)	Operations (Ops)
• Write code	• Deploy applications
• Build features	• Manage infrastructure
• Focus on application logic	• Ensure availability & performance
• Work in short iterations	• Handle monitoring, logging, backups
• Aim for fast delivery	• Keep the systems running smoothly

DevOps brings them together so that they work as one team with a common goal ☺

③ Traditional vs DevOps

	Traditional Model	DevOps Model
Approach	Siloed (Dev vs Ops)	Collaborative
Teams	Separate teams	Integrated teams
Release Cycle	Infrequent (months)	Frequent (days / hours)
Automation	Manual processes	Automated
Culture	Blame culture	Shared responsibility
Focus	Stability	Speed + Stability

"From Silos to Collaboration"

④ Goals of DevOps

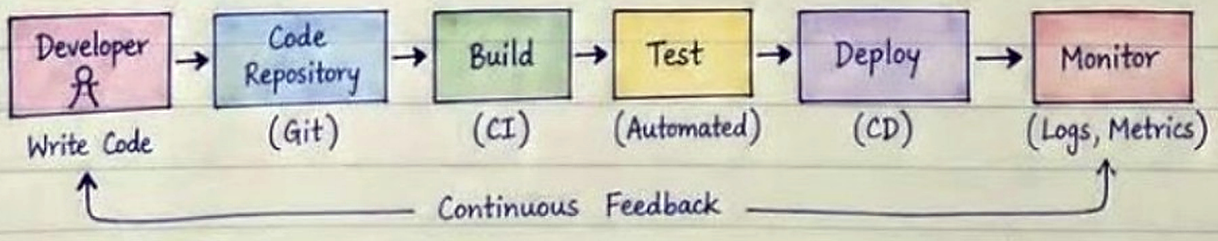
- ✓ Faster delivery of software
- ✓ High availability and reliability
- ✓ Automated and repeatable processes
- ✓ Continuous feedback and improvement
- ✓ Better collaboration between teams
- ✓ Scalability and efficiency

⑤ DevOps Culture

- ✓ Collaboration
- ✓ Automation
- ✓ Continuous Learning
- ✓ Transparency
- ✓ Shared Responsibility
- ✓ Focus on Customer Value

"Same Goal Same Team Better Software"

⑥ DevOps Workflow (High Level)



Automate everything that can be automated !

DEVOPS LIFECYCLE

"Continuous Collaboration for Faster Delivery"

"Smaller Changes Faster Value"

① What is DevOps Lifecycle ?

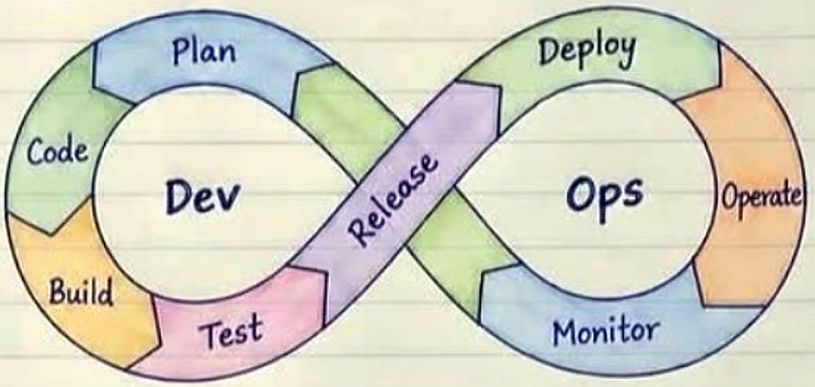
- DevOps lifecycle is a continuous and iterative process that brings together development and operations to deliver high-quality software faster.
- It focuses on automation, collaboration and continuous improvement.
- The lifecycle is often represented as an ∞ (infinity loop) showing that it is a continuous cycle, not a one-time process.

Key Idea

Plan, Build, Deploy, Operate and Monitor in a continuous loop to deliver better software.

② DevOps Lifecycle Stages

Collaboration
Feedback
Continuous
Improvement



"It's a cycle, not a phase."

③ Explanation of Each Stage

Stage	Description
1. Plan	Understand business requirements, plan features, create roadmap.
2. Code	Develop and commit code using version control (e.g., Git).
3. Build	Compile code, resolve dependencies, create build artifacts.
4. Test	Run automated tests (unit, integration, security tests) to ensure quality.
5. Release	Prepare the application for deployment (versioning, artifact storage).
6. Deploy	Deploy to production or staging environment (automated).
7. Operate	Manage infrastructure, keep the application running smoothly.
8. Monitor	Track performance, logs, metrics and get feedback for improvements.

④ Benefits of DevOps Lifecycle

- ✓ Faster time to market
- ✓ Improved collaboration between teams
- ✓ Higher software quality
- ✓ Automated and repeatable processes
- ✓ Better reliability and scalability
- ✓ Continuous feedback and improvement
- ✓ Reduced risk of failures
- ✓ Increased customer satisfaction

⑤ Remember !

- It's an iterative cycle (not linear).
- Automation is key at every stage.
- Feedback from monitoring goes back to planning.
- Culture, communication and collaboration are as important as tools.
- The goal is to deliver value continuously to the customer.

GIT & VERSION CONTROL

"Track Changes . Collaborate Better . Build Together"

"Good Developers Use Git, Great Developers Master it."

① What is Version Control ?

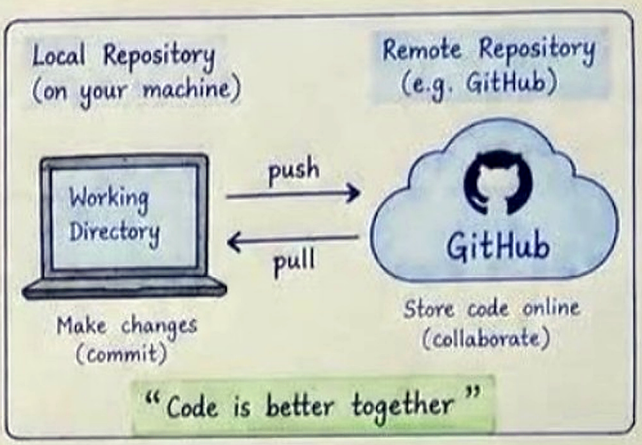
- Version Control System (VCS) helps to track changes in code over time.
- It allows multiple developers to work on the same project without overwriting each other's work.
- We can go back to previous versions if needed.
- It maintains a history of changes.

② Git vs GitHub

Git	GitHub
• Version control tool	• Cloud platform for Git
• Installed locally	• Hosts Git repositories
• Works offline	• Helps in collaboration
• Tracks changes in code	• Provides additional features (PR, Issues, etc.)

③ important Terms

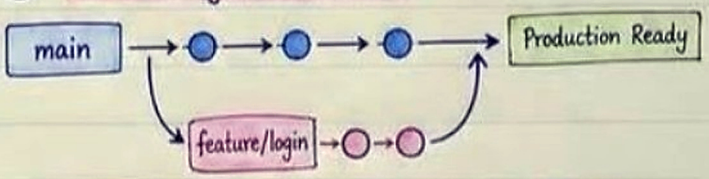
- **Repository** : A project folder tracked by Git.
- **Commit** : A snapshot of changes.
- **Branch** : A separate line of development.
- **Merge** : Combining changes from one branch to another.
- **Pull Request (PR)** : Request to merge changes (used in GitHub).
- **Clone** : Copy a remote repository to local machine.
- **Push** : Upload local changes to remote repository.
- **Pull** : Fetch latest changes from remote repository.



④ Basic Git Commands

Command	Description
git init	Initialize a new git repository
git clone <url>	Clone a remote repository
git status	Check status of changes
git add <file>	Stage changes
git commit -m "msg"	Commit changes with message
git push	Push changes to remote repository
git pull	Fetch and merge changes from remote
git branch	List/Create branches
git checkout <branch>	Switch to a branch
git merge <branch>	Merge a branch into current bramote

⑤ Branching Workflow



Create a branch → Make changes → Merge

⑥ Why Git is important ?

- ✓ Track history of changes
- ✓ Work in parallel with team members
- ✓ Safe and easy collaboration
- ✓ Helps in code review (via Pull Requests)
- ✓ Rollback to previous versions
- ✓ Widely used in industry
- ✓ Essential for DevOps, CI/CD, automation

"Commit today for a better tomorrow"

⑦ Tips & Best Practices

- ✓ Write meaningful commit messages
- ✓ Use branches (don't work directly on main)
- ✓ Keep commits small and focused
- ✓ Pull before you push
- ✓ Use .gitignore to exclude unnecessary files
- ✓ Follow a consistent branching strategy (e.g. Git Flow)

Quick Example

```
git clone https://github.com/user/repo.git
cd repo
git checkout -b feature/login
# make changes
git add .
git commit -m "Add login feature"
git push origin feature/login
```

"Small Consistent Commits Lead to Big Progress"

CI/CD PIPELINE

"Automate Today · Deliver Faster · Build Better"

"Automate the boring stuff, focus on what matters !"

① What is CI/CD ?

- CI/CD is a set of practices to automate the process of building, testing and deploying software.
- It helps in delivering changes to users faster and more reliably.
- It reduces manual effort and minimizes human errors.

Key Benefits

- ✓ Faster delivery
- ✓ Early bug detection
- ✓ Improved collaboration
- ✓ Reliable releases
- ✓ Reduced manual work
- ✓ Better product quality

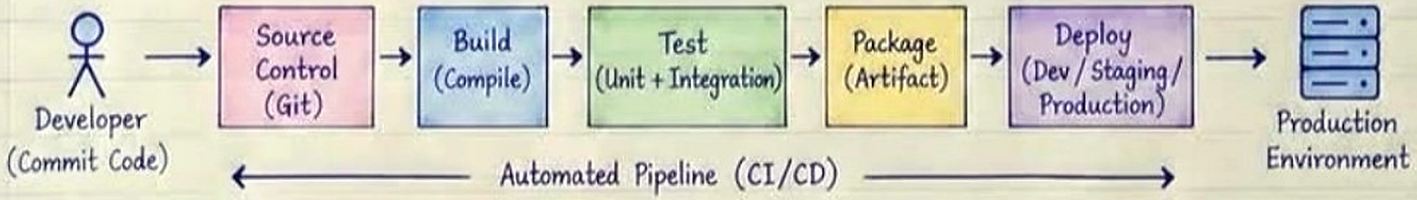
② CI vs CD

Aspect	Continuous Integration (CI)	Continuous Delivery (CD)
Purpose	Automatically build and test code changes	Automatically prepare the release for production
Focus	Integration & Testing	Deployment & Release
When	Every code commit	After successful build & test
Result	A tested build	A deployable release

Continuous Deployment (Advanced)

In Continuous Deployment, every change that passes the tests is automatically deployed to production (without manual approval).

③ Typical CI/CD Pipeline



④ Pipeline Stages (Detailed)

1. **Source** : Code is committed to a version control system (e.g., Git).
2. **Build** : The code is compiled and dependencies are installed.
3. **Test** : Automated tests (unit, integration, security) are executed.
4. **Package** : Build artifacts are created (e.g., .jar, .war, Docker image).
5. **Deploy** : Artifact is deployed to staging / production environment.
6. **Monitor** : Application is monitored for performance and errors.
7. **Feedback** : Feedback is collected for continuous improvement.

⑤ Popular CI/CD Tools

- Jenkins
- GitHub Actions
- GitLab CI/CD
- Azure DevOps
- CircleCI
- Travis CI
- AWS CodePipeline
- Bamboo

"Tools may change, but the principles remain the same."

⑥ CI/CD Best Practices

- ✓ Keep the pipeline simple and modular.
- ✓ Run automated tests at every stage.
- ✓ Use infrastructure as code (IaC).
- ✓ Keep secrets secure (use secret managers).
- ✓ Deploy to a staging environment first.
- ✓ Monitor applications after deployment.
- ✓ Implement rollback strategy.

⑦ Interview Question

Q: What is the difference between Continuous Delivery and Continuous Deployment?

Ans: In Continuous Delivery, the code is automatically tested and prepared for release, but deployment to production is done manually. In Continuous Deployment, every change that passes the tests is automatically deployed to production.

JENKINS

"Build · Automate · Deliver"

"Automate the repetitive, Focus on what matters"

① What is Jenkins ?

- Jenkins is an open-source automation server used to implement CI/CD pipelines.
- It helps in automating the building, testing and deploying of software.
- Supports a wide range of plugins and integrates with many tools (Git, Docker, Kubernetes, etc.).
- Written in Java and maintained by the Jenkins community.

Why Jenkins ?

- ✓ Open source and free
- ✓ Easy to set up and use
- ✓ Large plugin ecosystem
- ✓ Supports distributed builds
- ✓ Integrates with almost all DevOps tools
- ✓ Active community support
- ✓ Highly extensible

② Jenkins Architecture

Jenkins

Jenkins Controller (Master)

- Schedules jobs
- Manages plugins
- Stores configuration
- Distributes work to agents

→ Agent (Node 1) Build/Test/Deploy

→ Agent (Node 2) Build/Test/Deploy

→ Agent (Node 3) Build/Test/Deploy

"Controller manages, Agents execute"

③ Key Terms

Term	Description
Job	A task to be executed (build, test, deploy, etc.)
Build	An execution of a job.
Pipeline	Defines the build process as code (Jenkinsfile).
Node / Agent	Machine that executes the job.
Plugin	Adds extra features (e.g., Git, Docker, Kubernntes).
Jenkinsfile	A text file that defines the pipeline.
Workspace	Directory where the job is executed.
Build Trigger	Event that starts the build (e.g., code push).

④ Types of Jobs

- Freestyle Project – Simple job configuration via UI.
- Pipeline Project – Uses Jenkinsfile (recommended).
- Multibranch Pipeline – Automatically builds multiple branches.
- Folder – Organize jobs into folders.

⑤ Jenkins Pipeline (Example)

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying to production...'
            }
        }
    }
}
```

"Pipeline as Code = Repeatable + Reliable"

⑥ Jenkins Pipeline Flow

Code Push (Git) → Trigger (Build) → Execute Pipeline → Build & Test → Deploy to Env

⑦ Important Jenkins Concepts

- ✓ Use Jenkinsfile for version-controlled pipelines.
- ✓ Keep the pipeline modular and reusable.
- ✓ Use credentials for sensitive data (passwords, tokens).
- ✓ Leverage plugins to integrate with tools.
- ✓ Use agents for distributed builds.
- ✓ Monitor build status and set up notifications (email/Slack).
- ✓ Clean workspace after builds to save space.

Date: ___/___/____

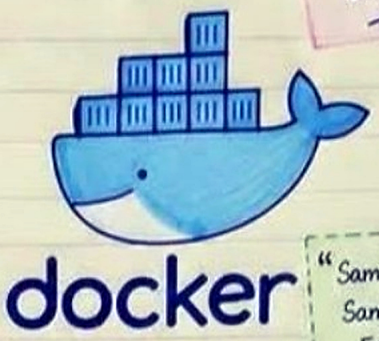
DOCKER

"Build Once · Run Anywhere"

"Containerize Today Deploy Everywhere"

① What is Docker ?

- Docker is an open-source platform that allows us to develop, ship and run applications in containers.
- It packages the application and its dependencies together so that it can run consistently in any environment.
- Docker uses containerization to isolate applications from each other and from the host system.
- It makes application deployment faster, easier and more reliable.



"Same Application Same Behavior Everywhere"

② Containers vs Virtual Machines

Feature	Containers (Docker)	Virtual Machines (VM)
Architecture	Share host OS kernel	Have own OS (Guest OS)
Size	Lightweight (MBs)	Heavy (GBs)
Boot time	Seconds	Minutes
Performance	Near native	Slower
Isolation	Process level isolation	Full OS isolation
Resource usage	Less	More
Use case	Microservices, scalable apps	Legacy applications

③ Key Concepts

- **Image** : Read-only template to create containers.
- **Container** : A running instance of an image.
- **Dockerfile** : A text file with instructions to build an image.
- **Docker Registry** : Stores Docker images (e.g., Docker Hub).
- **Docker Hub** : Public registry provided by Docker.
- **Volume** : Persists data outside the container.
- **Network** : Helps containers communicate with each other.

Analogy :
Docker image is like a blueprint and container is the actual house built from that blueprint.

④ How Docker Works ?

(Instructions) (Read-only template) (Running instance)

⑤ Basic Docker Commands

Command	Description
`docker --version`	Check Docker version
`docker pull <image>`	Download image from registry
`docker images`	List all images
`docker run <image>`	Run a container from image
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <container>`	Stop a running container
`docker start <container>`	Start a stopped container
`docker rm <container>`	Remove a container
`docker rmi <image>`	Remove an image
`docker logs <container>`	View container logs
`docker exec -it <container> bash`	Access container shell

⑥ Dockerfile Example

```
1  FROM node:18                  # Base image
2  WORKDIR /app                  # Set working directory
3  COPY package*.json ./         # Copy dependency files
4  RUN npm install               # Install dependencies
5  COPY . .                      # Copy project files
6  EXPOSE 3000                   # Expose port
7  CMD ["npm", "start"]          # Start application
```

⑦ Benefits of Docker

- ✓ Consistency across different environments
- ✓ Faster deployment and rollback
- ✓ Lightweight and portable
- ✓ Useful for microservices architecture
- ✓ Better resource utilization
- ✓ Easy scaling and versioning

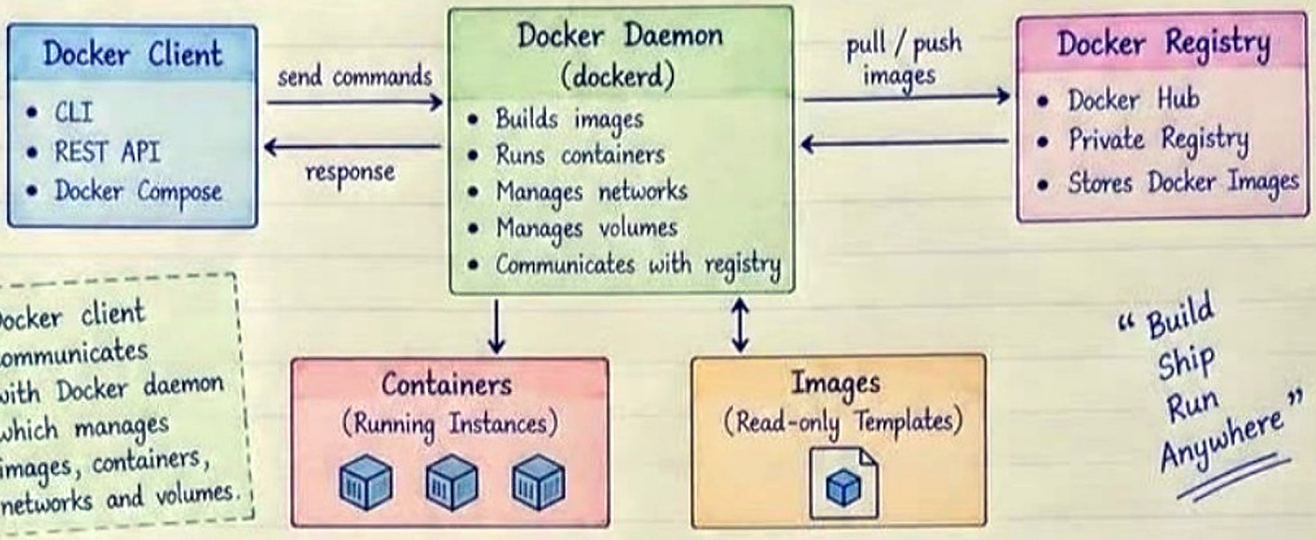
"Build Ship Run Repeat"

DOCKER ARCHIECTURE

"Understand the Building Blocks"

"Containers Make Deployment Simple"

1 High Level Architecture



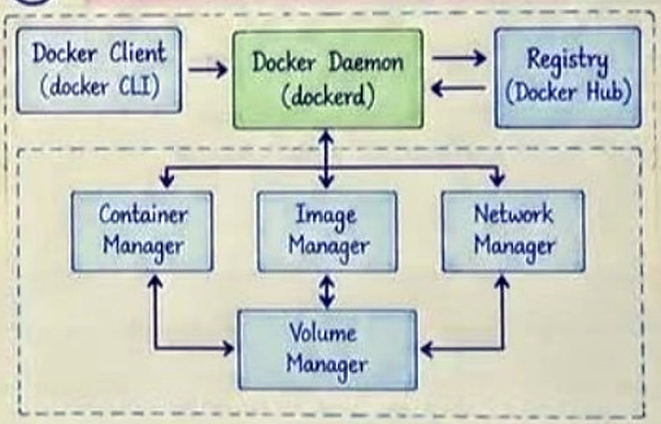
"Docker client communicates with Docker daemon which manages images, containers, networks and volumes."

"Build Ship Run Anywhere"

2 Components Explained

Component	Description
Docker Client	Used to interact with Docker (CLI or API).
Docker Daemon	Core service that builds, runs and manages containers, images, networks and volumes.
Docker Image	Read-only template used to create containers.
Docker Container	Running instance of an image.
Docker Registry	Stores images (e.g., Docker Hub).
Dockerfile	Text file with instructions to build an image.
Volumes	Persist data outside containers.
Networks	Enable communication between containers.

3 Docker Architecture in Detail



4 Docker Data Flow (Example)

1. Write a Dockerfile with application setup.
2. Build the image using docker build.
3. Docker daemon creates the image.
4. Run a container from the image.
5. Container uses networks and volumes.
6. Pull/Push images from/to registry.
7. Access the running application.

5 Docker Network Types

Network Type	Use Case
bridge	Default network, used for single host.
host	Container uses host's network directly.
none	No network access.
overlay	Used in multi-host environments (Swarm/K8s).
macvlan	Assign MAC address to container.

6 Docker Volumes

- Volumes persist data outside the container.
- Useful for databases and stateful applications.
- Can be created using: docker volume create <name>
- Mount using: -v <name>:<path> or --mount

7 Key Takeaways

- ✓ Docker separates application from infrastructure.
- ✓ Images are immutable, containers are ephemeral.
- ✓ Use volumes for persistent data.
- ✓ Use networks for communication.
- ✓ Docker architecture is simple but powerful.

"Same Image Same Behavior Everywhere"

KUBERNETES

"Container Orchestration for a Better Tomorrow"

"Orchestrate Containers Scale Effortlessly Stay Resilient"

1 What is Kubernetes ?

- Kubernetes (K8s) is an open-source container orchestration platform.
- It automates deployment, scaling, management and networking of containerized applications.
- It was originally developed by Google and is now maintained by the CNCF (Cloud Native Computing Foundation).
- Kubernetes helps run and manage containers at scale across a cluster of machines.

2 Why Kubernetes ?

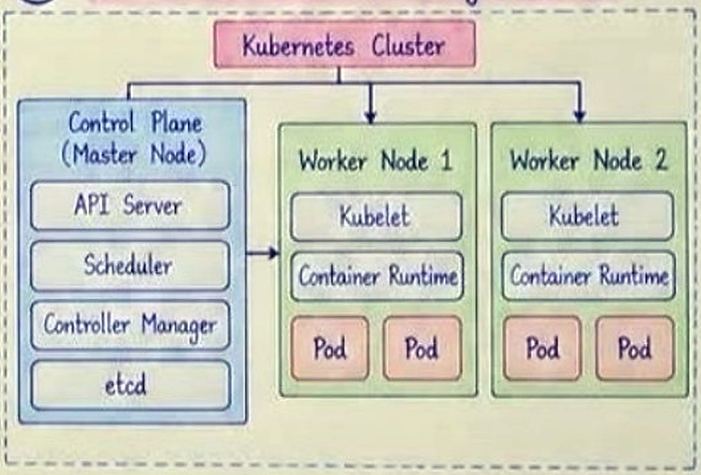
- ✓ Automates deployment and scaling
- ✓ Self-healing (restarts failed containers)
- ✓ Load balancing and service discovery
- ✓ Efficient resource utilization
- ✓ Supports rolling updates and rollbacks
- ✓ Works across cloud, on-prem and hybrid
- ✓ Large community and ecosystem
- ✓ Industry standard for container orchestration

"From Containers to Production Kubernetes Makes it Happen"

3 Key Concepts

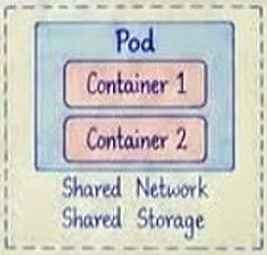
Concept	Description
Cluster	A set of machines that run Kubernetes.
Node	A worker machine (VM or physical) in the cluster.
Pod	Smallest deployable unit (one or more containers).
Deployment	Manages replica pods and updates.
Service	Provides stable network access to pods.
Namespace	Logical isolation within a cluster.
ConfigMap	Stores non-sensitive configuration data.
Secret	Stores sensitive data (passwords, tokens, etc.)
Ingress	Manages external access (HTTP/HTTPS).
Persistent Volume (PV)	Provides persistent storage
Persistent Volume Claim (PVC)	Request for storage from PV.

4 Kubernetes Cluster (High Level)



5 Pod

- A pod is the smallest deployable unit in Kubernetes.
- It can contain one or more containers (usually one).
- Pods share the same IP address, storage and network namespace.
- Pods are ephemeral (can be created, destroyed, restarted).

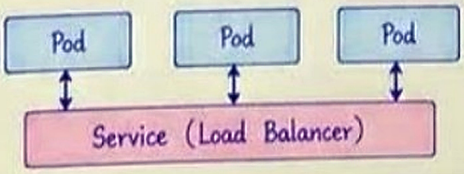


6 Deployment

- Deployment manages a set of replica pods.
- Ensures desired number of pods are always running.
- Supports rolling updates and rollbacks.
- Helps in zero-downtime deployments.

7 Service

- A Service provides a stable IP and DNS name to access a set of pods.
- It acts as a load balancer between pods.
- Types : ClusterIP (default), NodePort, LoadBalancer, ExternalName.



"Pods are ephemeral, Services give stability"

8 Other Components

Component	Description
Namespace	Isolates resources within a cluster.
ConfigMap	Stores configuration data.
Secret	Stores sensitive data.
Ingress	Manages external HTTP/HTTPS access.
PV	Persistent storage in the cluster.
PVC	Request for persistent storage.
Helm	Package manager for Kubernetes applications.
Kubectl	CLI tool to interact with Kubernetes.

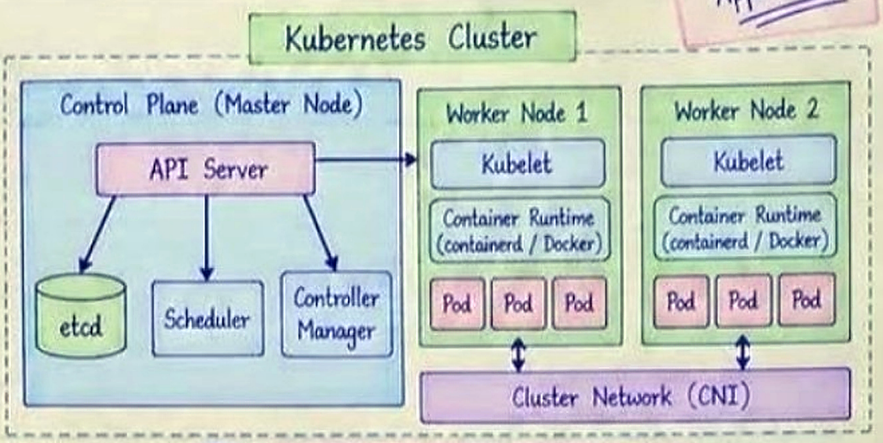
K8s ARCHITECTURE

"Many Nodes · One Cluster · Infinite Possibilities"

"Abstract Infrastructure Manage Applications"

1 Overview

- Kubernetes follows a master-worker architecture (now called control plane and worker nodes).
- Control plane manages the cluster.
- Worker nodes run the actual applications (containers in pods).
- All components communicate via the Kubernetes API Server.



2 Control Plane Components

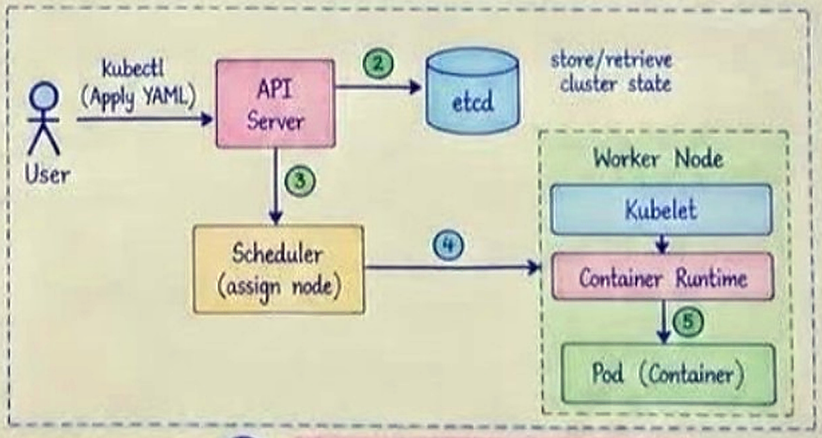
Component	Description
API Server	• Entry point for all requests (REST API). • Validates and processes requests. • Communicates with other components.
etcd	• Distributed key-value store. • Stores all cluster data (desired state). • Highly consistent and fault tolerant.
Scheduler	• Assigns pods to suitable worker nodes. • Considers resource requirements, constraints and policies.
Controller Manager	• Runs controller loops (e.g., Deployment, ReplicaSet, Node, etc.) • Ensures actual state matches desired state.

3 Worker Node Components

Component	Description
Kubelet	• Runs on each worker node. • Communicates with API Server. • Ensures containers in pods are running as expected.
Container Runtime	• Responsible for running containers. • Examples: containerd (default), Docker, CRI-O.
Kube-proxy	• Manages network rules on the node. • Enables communication between pods and services.
Pod	• Smallest deployable unit. • Runs one or more containers. • Has shared network and storage.

4 Communication Flow

1. User sends request (kubectl / UI).
2. Request goes to API Server.
3. API Server validates and stores data in etcd.
4. Scheduler assigns pod to a worker node.
5. Kubelet on the node pulls the image and starts the container using container runtime.
6. Kube-proxy sets up networking rules.
7. Application is running and accessible via Service.



5 Key Takeaways

- ✓ Kubernetes separates control plane and worker nodes.
- ✓ API Server is the brain of the cluster.
- ✓ etcd stores the desired state.
- ✓ Scheduler decides where pods run.
- ✓ Kubelet ensures pods are always running.
- ✓ Kubernetes is declarative (you declare the desired state).

"You don't manage servers, You manage workloads"

6 Common Tools & Add-ons

- Kubectl - CLI tool to interact with cluster
- Kubeadm - Setup Kubernetes clusters
- Helm - Package manager for K8s
- CNI - Container networking (Calico, Flannel, etc.)
- Ingress Controller - Manage external access
- Metrics Server - Collects resource metrics
- Dashboard - Web UI for Kubernetes

KUBERNETES DEPLOYMENT

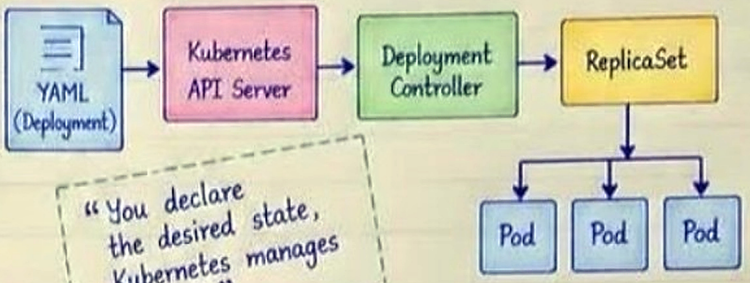
"From Code to Containers to Production"

"Define Deploy Scale Repeat"

① What is Deployment ?

- Deployment is a Kubernetes resource used to manage stateless applications.
- It ensures the desired number of pod replicas are always running.
- It handles rolling updates, rollbacks and self-healing automatically.
- You define the desired state in a YAML file and Kubernetes makes it happen.

② Deployment Workflow



"You declare the desired state, Kubernetes manages the rest"

③ Deployment YAML Example

```yaml
apiVersion: apps/v1      # API version for Deployment
kind: Deployment         # Resource type
metadata:
  name: my-app           # Deployment name
  labels:
    app: my-app
spec:
  replicas: 3            # Number of pod replicas
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: nginx:latest  # Container image
        ports:
        - containerPort: 80  # Expose port
```

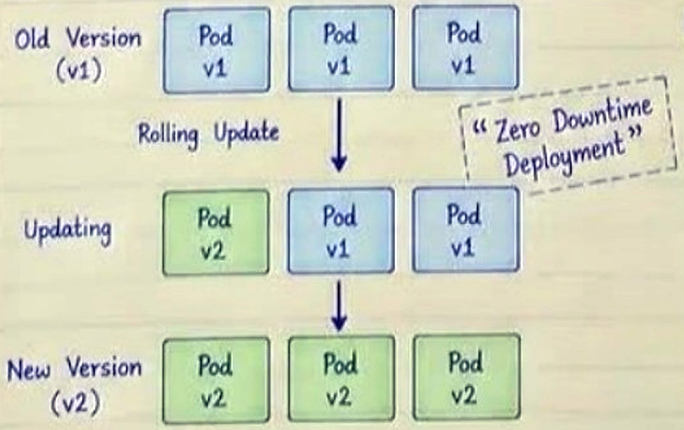
④ Update Strategies

Strategy	Description	Use Case
RollingUpdate (default)	Gradually replaces old pods with new ones.	Most common, zero downtime
Recreate	Terminates all old pods before creating new ones.	Simple apps, not sensitive to downtime.

⑤ Useful Commands

Command	Description
kubectl create -f deployment.yaml	Create deployment
kubectl get deployments	List deployments
kubectl get pods	List running pods
kubectl describe deployment <name>	Get detailed info
kubectl logs <pod-name>	View logs
kubectl set image deployment/<name> <container>=<image>	Update image (rolling update)
kubectl rollout status deployment/<name>	Check rollout status
kubectl rollout undo deployment/<name>	Rollback to previous version
kubectl delete deployment <name>	Delete deployment

⑥ Rolling Update (How it works)



"Zero Downtime Deployment"

⑦ Key Takeaways

- ✓ Deployment manages stateless applications.
- ✓ Kubernetes maintains desired state automatically.
- ✓ Rolling updates enable zero downtime.
- ✓ Use YAML files to define deployments.
- ✓ Monitor rollout status and be ready to rollback.
- ✓ Combine with Services for stable access.
- ✓ Keep your images small and secure.
- ✓ Practice deploying your own app !

"Small Steps to Big Deployments"

Date: __/__/____

KUBERNETES SERVICES

"Expose · Connect · Scale"

"A Service Gives Stability to a Changing World"

① What is a Service ?

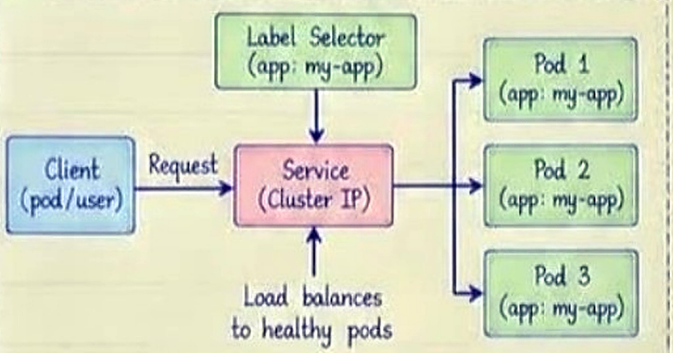
- A Service is a stable network endpoint that exposes a set of Pods.
- Since pods are ephemeral (IP changes), Service provides a consistent IP and DNS name.
- It load balances traffic across healthy pods.
- It works with a label selector to find and route traffic to pods.

② Why do we need Services ?

- ✓ Pods can die and get new IPs.
- ✓ Provides a stable endpoint (IP + DNS).
- ✓ Load balances traffic across multiple pods.
- ✓ Decouples the client from the application pods.
- ✓ Enables internal and external access.
- ✓ Supports service discovery in the cluster

"Pods Change Services Stay"

③ How does a Service work ?



④ Types of Services

Type	Description	Use Case
ClusterIP	Exposes service inside cluster (default).	Internal communication between services.
NodePort	Exposes service on each node's IP at a static port.	Access from outside cluster (dev/testing).
LoadBalancer	Creates an external load balancer (cloud provider).	Production applications on cloud (AWS, GCP, etc).
ExternalName	Maps service to an external DNS name.	Access external services (e.g., database, API)

⑤ ClusterIP (Default)

- Exposes the service on an internal IP.
- Only accessible within the cluster.
- Most commonly used type.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 8080
```

"Simple and Powerful"

⑥ NodePort

- Exposes the service on each node's IP at a static port (30000-32767).
- Useful for local testing or on-prem.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30007
```

"Access Your App on Node IP"

⑦ LoadBalancer

- Creates an external load balancer (in cloud environments).
- Automatically provisions a load balancer and assigns a public IP.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 8080
```

"Perfect for Production"

⑧ Service Discovery (DNS)

- Kubernetes assigns a DNS name to each service.
- Format: <service-name>.<namespace>.svc.cluster.local
- Example: my-service.default.svc.cluster.local
- Pods can communicate using the service name.

⑨ Key Takeaways

- ✓ Services provide stable endpoints.
- ✓ ClusterIP for internal access.
- ✓ NodePort for testing or on-prem.
- ✓ LoadBalancer for production (cloud).
- ✓ Services enable service discovery.
- ✓ Use labels and selectors to connect pods.
- ✓ Choose the right type based on use case.

"Right Service Right Access Bigger Possibilities"

KUBERNETES NETWORKING

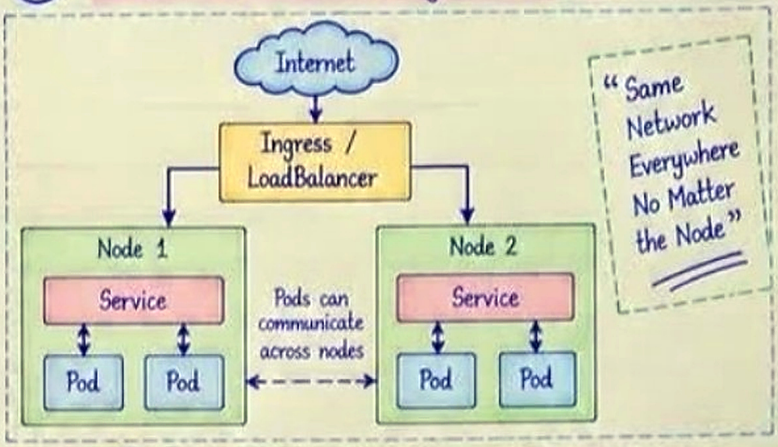
"Connect Pods · Expose Applications · Enable Communication"

"Networking Makes Everything Work Together"

1 Why Networking in Kubernetes ?

- Enables communication between Pods, Services and external world.
- Each Pod gets a unique IP address.
- Kubernetes provides a flat network model (all pods can communicate across nodes).
- Services provide stable endpoints for pods.
- Supports different exposure types (ClusterIP, NodePort, LoadBalancer, Ingress).

2 Kubernetes Networking Model



"Same Network Everywhere No Matter the Node"

3 Networking Building Blocks

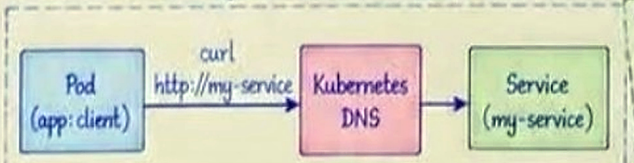
Component	Description
Pod	Gets a unique IP address.
Service	Provides a stable endpoint to access pods.
Ingress	Manages external HTTP/HTTPS access.
CNI (Container Network Interface)	Provides the actual networking implementation (e.g., Calico, Flannel, Weave).
Kube-proxy	Maintains network rules and load balances traffic to pods.
DNS	Provides internal name resolution (e.g., <service-name>.<namespace>.svc.cluster.local).

4 Service Types (Networking)

Type	Description	Use Case
ClusterIP	Internal IP, accessible only within cluster.	Default type, used for internal communication.
NodePort	Exposes service on a static port on each node.	Useful for testing or external access (dev).
LoadBalancer	Creates an external load balancer (cloud provider).	Used in production environments.
ExternalName	Maps service to an external DNS name.	Access external services (e.g., database, API).

5 DNS in Kubernetes

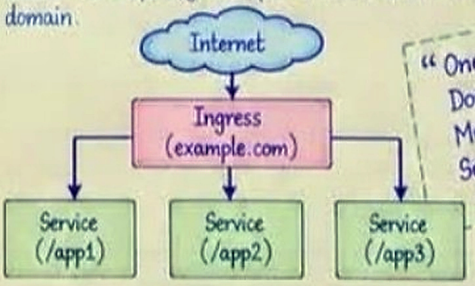
- Kubernetes provides internal DNS for service discovery.
- Format: <service-name>.<namespace>.svc.cluster.local
- Example: my-service.default.svc.cluster.local
- Pods can directly use the service name to communicate.



"No need to remember IPs, just use service name"

6 Ingress (HTTP/HTTPS)

- Ingress manages external access to services.
- Supports host-based and path-based routing.
- Requires an Ingress Controller (e.g., NGINX, Traefik).
- Useful for exposing multiple services under a single domain.



"One Domain Multiple Services"

7 CNI Plugins (Examples)

CNI Plugin	Description
Calico	Popular, supports network policies.
Flannel	Simple and easy to set up.
Weave Net	Easy to use, good for small clusters.
Cilium	Advanced, uses eBPF, supports security.

8 Key Takeaways

- ✓ Kubernetes provides a flat network model.
- ✓ Services abstract pod IPs and provide stable access.
- ✓ Ingress enables external access with routing.
- ✓ CNI plugins handle the actual networking.
- ✓ Use DNS for service discovery.
- ✓ Choose the right service type based on your use case.

"Right Networking Builds Scalable Applications"

KUBERNETES STORAGE

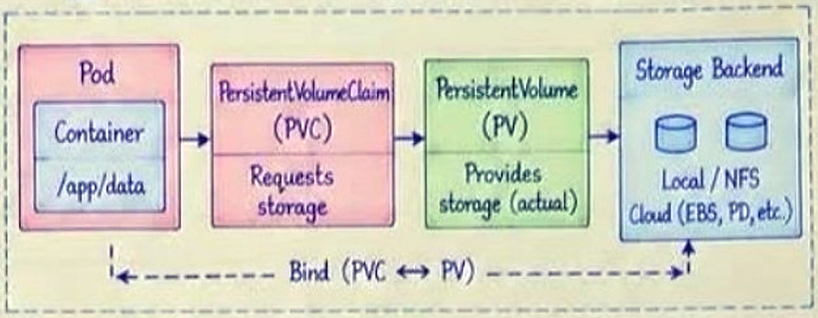
"Persistent Data · Stateful Apps · Beyond Containers"

"Data Survives Containers"

1 Why Storage in Kubernetes ?

- By default, containers are ephemeral (data is lost when pod is deleted).
- Kubernetes provides persistent storage to keep data beyond pod lifecycle.
- Useful for stateful applications (databases, message queues, etc.).
- Storage is abstracted using PersistentVolumes and PersistentVolumeClaims.

2 Kubernetes Storage Architecture



3 Key Concepts

Concept	Description
PersistentVolume (PV)	A piece of storage in the cluster (provisioned by admin or dynamically).
PersistentVolumeClaim (PVC)	A request for storage by a user
StorageClass	Defines how storage should be provisioned (dynamic provisioning).
Access Modes	Defines how the volume can be mounted (RWO, ROX, RWX)
Reclaim Policy	Defines what happens to the PV after PVC is deleted (Retain, Delete, Recycle).
Volume	Mounted inside a pod using a volume which refers to a PVC.

4 Access Modes

Mode	Description
ReadWriteOnce (RWO)	Can be mounted as read-write by a single node
ReadOnlyMany (ROX)	Can be mounted read-only by multiple nodes
ReadWriteMany (RWX)	Can be mounted as read-write by multiple nodes (used with NFS, etc.).

5 Reclaim Policy

Policy	Description
Retain	PV is retained even after PVC is deleted (manual cleanup).
Delete	PV is automatically deleted.
Recycle	(Deprecated) Basic scrub and reuse.

6 Example: PVC and Pod

PersistentVolumeClaim (pvc.yaml)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: standard
```

Pod using PVC (pod.yaml)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: data
          mountPath: /app/data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: my-pvc
```

7 StorageClass (Dynamic Provisioning)

StorageClass (sc.yaml)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs  # or gce, azure, etc.
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

"Dynamic Provisioning Makes Storage Easy"

8 Key Takeaways

- ✓ Use PVC for persistent storage.
- ✓ Use StorageClass for dynamic provisioning.
- ✓ Choose appropriate access modes.
- ✓ Understand reclaim policies.
- ✓ Storage enables stateful applications in Kubernetes.
- ✓ Works with cloud providers (AWS, GCP, Azure) and on-prem (NFS, etc.).

"Stateful Apps Need Persistent Storage"

KUBERNETES SECURITY, MONITORING & SCALING

"Run Securely · Observe Clearly · Scale Effortlessly"

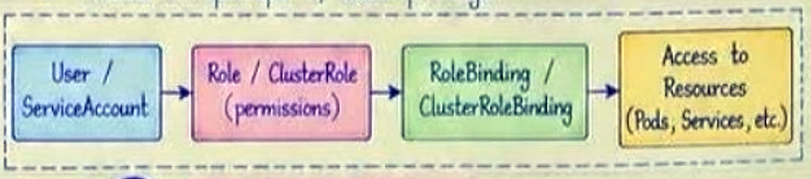
"Secure Observe Scale Succeed"

1 Why Security Matters ?

- Kubernetes runs critical applications.
- Multiple users and services share the cluster.
- Security ensures isolation, access control and protection from threats.
- Misconfiguration can expose sensitive data.
- Use built-in Kubernetes features like RBAC, Secrets, Network Policies, etc.

2 Authentication & Authorization (RBAC)

- Authentication verifies who you are (User, ServiceAccount).
- Authorization decides what you can do (using RBAC).
- RBAC uses Roles and RoleBindings to grant permissions.
- Follows the principle of least privilege.



3 Kubernetes Secrets

- Used to store sensitive data (passwords, API keys, tokens).
- Stored in base64 encoded format (not encrypted by default).
- Can be mounted as environment variables or files.
- For production, use tools like HashiCorp Vault, AWS KMS, Azure Key Vault, etc.

Secret Example (secret.yaml)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  DB_PASSWORD: bXlyYXNZd29yZA==   # base64 encoded
```

"Keep Secrets Safe"

4 Network Policies

- Control the traffic between pods.
- Define which pods can communicate with each other.
- Helps in restricting unauthorized access.
- Works at the network layer using labels and selectors.

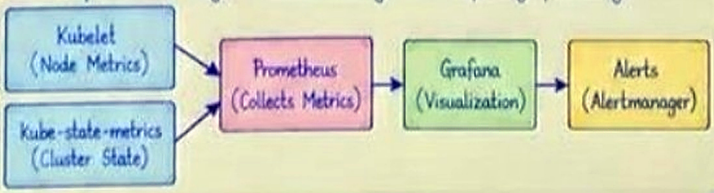
NetworkPolicy Example (allow-frontend.yaml)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: backend
```

"Restrict Access Build Safer Clusters"

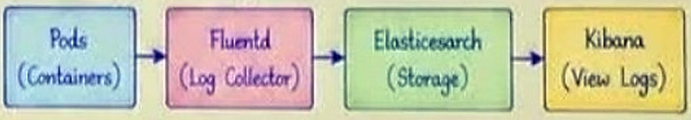
5 Monitoring in Kubernetes

- Monitoring helps you understand the health and performance of your cluster and applications.
- Popular tools: Prometheus, Grafana, Kubernetes Dashboard.
- Metrics collected from nodes, pods, services, etc.
- Helps in alerting, troubleshooting and capacity planning.



6 Logging in Kubernetes

- Logs help in debugging and understanding application behavior.
- Common stack: EFK (Elasticsearch + Fluentd + Kibana) or ELK (Elasticsearch + Logstash + Kibana).
- Logs can be collected from containers, system components, and application logs.



7 Scaling in Kubernetes

- Scale your applications based on demand.
- Types of scaling:
  - Manual Scaling → kubectl scale deployment <name> --replicas=N
  - Horizontal Pod Autoscaler (HPA) → scales based on CPU/Memory
  - Vertical Pod Autoscaler (VPA) → adjusts resource requests/limits
  - Cluster Autoscaler → adds/removes nodes automatically

8 Key Takeaways

- ✓ Use RBAC for fine-grained access control.
- ✓ Store sensitive data using Secrets.
- ✓ Implement Network Policies to restrict traffic.
- ✓ Monitor your cluster using Prometheus & Grafana.
- ✓ Set up logging for better observability.
- ✓ Use autoscaling to handle varying loads.
- ✓ Follow security best practices and least privilege.

"A Secure Observable & Scalable Kubernetes is a Happy Kubernetes"

KUBERNETES - CHEAT SHEET & FINAL NOTES

"Small Commands • Big Impact • You've Got This!"

"Practice Learn Deploy Grow"

1 Most Useful Kubectl Commands

Command	Description
kubectl get pods	List all pods
kubectl get svc	List all services
kubectl get nodes	List all nodes
kubectl get deployments	List all deployments
kubectl describe pod <name>	Get details of a pod
kubectl logs <pod-name>	View pod logs
kubectl exec -it <pod> -- bash	Access pod shell
kubectl apply -f <file.yaml>	Create/update resources
kubectl delete -f <file.yaml>	Delete resources
kubectl rollout status deployment/<name>	Check rollout status
kubectl get events	View cluster events
kubectl config get-contexts	List contexts
kubectl config use-context <name>	Switch context

2 Common YAML Templates

Pod (pod.yaml)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
  - name: app
    image: nginx
```

Service (service.yaml)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 80
```

"YAML + Kubectl = Kubernetes in Action"

Deployment (deployment.yaml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: app
        image: nginx
```

ConfigMap (configmap.yaml)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  ENV: production
  APP_NAME: my-app
```

3 Kubernetes Troubleshooting

- Pod not starting? → Check logs and events.
- CrashLoopBackOff? → Verify configuration and env vars.
- Service not accessible? → Check selector and endpoints.
- ImagePullError? → Verify image name and registry access.
- Node not ready? → Check node status and system logs.
- Performance issues? → Check resource usage (CPU/Memory)

4 Best Practices

- ✓ Use meaningful names and labels.
- ✓ Keep images small and secure.
- ✓ Use resource requests and limits.
- ✓ Separate configs using ConfigMaps and Secrets.
- ✓ Use RBAC for proper access control.
- ✓ Enable monitoring and logging.
- ✓ Follow the principle of least privilege.
- ✓ Keep your manifests in version control (Git)

"Observe Debug Fix Learn Repeat"

5 Real World Use Cases

- Deploying web applications (frontend + backend)
- Running microservices at scale.
- Handling batch jobs and cron jobs.
- Managing stateful applications (databases).
- Auto scaling based on traffic.
- Blue-Green and Canary deployments.
- Multi-environment setup (dev, staging, prod)

"From Idea to Production Kubernetes Powers It"

6 Kubernetes Ecosystem (Recap)

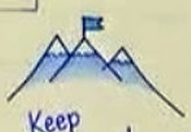
Tool	Purpose
kubectl	CLI to interact with cluster
kubeadm	Set up Kubernetes cluster
Helm	Package manager for Kubernetes
Prometheus	Monitoring and metrics
Grafana	Visualization and dashboards
Ingress Controller	Manage external access (NGINX, Traefik)
CSI	Container Storage Interface
CNI	Container Network Interface

"Ecosystem Makes Kubernetes Even Stronger"

7 Final Takeaways

- ✓ Kubernetes helps you deploy, scale and manage applications efficiently.
- ✓ It abstracts the underlying infrastructure and gives you powerful tools.
- ✓ Start small, practice with real projects.
- ✓ Explore advanced topics like Operators, Service Mesh, and GitOps.
- ✓ Keep learning → the cloud native world is evolving fast!

"Containers Run Apps, Kubernetes Runs the Future!"



Keep Learning! Keep Building!